

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1704

COURSE OUTCOME COVERED

***CO4:** Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2 Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

*I **MITHRA DEVI S - 192524203** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Mithra Devi

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ). (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understandin g of Concepts	25	Demonstrates thorough understandin g and effectively integrates multiple concepts/tech nol ogies	Good understandi ng with proper integration of most concepts	Basic understandin g; limited or partial integration	Poor understa ndi ng; unable to integrate concepts
Experiment al Design	30	Well-planned, systematic implementati on with optimal solution	Correct implementati on with minor issues	Basic implementati on with errors or inefficiencies	Incorrect or incomplet e implemen tat ion
Use of Tools	20	Effective and advanced use of tools/technolo gi es with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Document ati on	10	Clear, well structured documentatio n with diagrams, code, and results	Organized documentati on with necessary details	Basic documentati on with missing details	Poor or incomplet e document ati on

EXPERIMENT 1 - Decision Tree Classification

(a) Load, pre-process, split

Dataset: the Iris dataset (150 samples, 4 numeric features: sepal length/width, petal length/width; target = species, 3 classes).

sepal length	sepal width	petal length	petal width	target	species
5.1	3.5	1.4	0.2	0	setosa
4.9	3.0	1.4	0.2	0	setosa
4.7	3.2	1.3	0.2	0	setosa
4.6	3.1	1.5	0.2	0	setosa
5.0	3.6	1.4	0.2	0	setosa

Data types:

all 4 features are float64; target is int64 (already label-encoded 0/1/2 by scikitlearn); species is the human-readable string label. Missing-value check: 0 missing values in every column — no imputation needed for this dataset.

Preprocessing note:

Iris ships fully numeric with no missing data, so the encoding/imputation steps requested by the brief have nothing to act on here; if a categorical feature or missing values were present (as in a Play-Tennis-style dataset), the same pipeline would add `pandas.get_dummies()/LabelEncoder` for categoricals and `SimpleImputer` for missing values before the split below.

Split: 70% train / 30% test, stratified by class → Train = 105 rows, Test = 45 rows.

(b) Build the Decision Tree and justify the root node

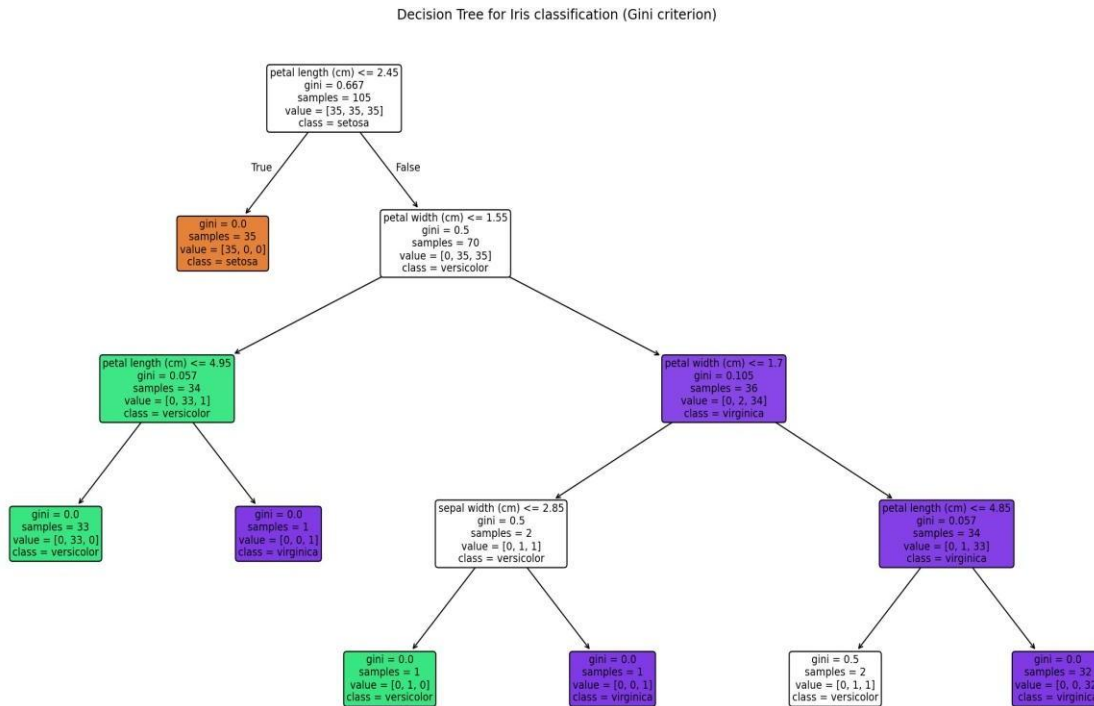


Fig. 1 — Trained Decision Tree (Gini criterion, max_depth=4)

Root split: petal length (cm) ≤ 2.45

Root Gini impurity = 0.6667 $\rightarrow$ Left child Gini = 0.0000 Right child Gini = 0.5000

Justification: the root is chosen by the Gini index (equivalently, Information-Gain-style impurity reduction) — among all 4 features and all possible thresholds, splitting on petal length ≤ 2.45 gives the single largest impurity drop: it perfectly isolates all 35 setosa samples into a pure leaf (Gini = 0, 100% setosa) in one step, cutting the parent's Gini from 0.667 to an average child Gini of just 0.25. Feature importances computed by the fitted tree confirm this numerically: petal length = 0.549 (highest), petal width = 0.436, sepal width = 0.014, sepal length = 0.000 — petal length is by far the most discriminative feature, which is exactly why the algorithm selects it as the root.

(c) Evaluation

Metric	Value
Accuracy	0.8889 (40/45 correct)

Confusion matrix (rows = actual, columns = predicted; order: setosa, versicolor, virginica):

		Pred: setosa		Pred: versicolor		Pred: virginica			
Actual: setosa		15		0		0			
Actual: versicolor		0		12		3			
Actual: virginica		0		2		13			
Class		Precision		Recall		F1-score		Support	
setosa		1.00		1.00		1.00		15	
versicolor		0.86		0.80		0.83		15	
virginica		0.81		0.87		0.84		15	

Comment on performance: *setosa* is separated perfectly (linearly separable from the other two species on petal measurements alone, matching the pure root split above). All 5 errors are *versicolor* $\leftrightarrow$ *virginica* confusions — these two species genuinely overlap in petal length/width, so this is expected, benign error rather than a modelling flaw; a shallow, 4-level tree still resolves 40/45 test cases correctly.

At least two misclassified instances (from the actual test run):

- Row (petal length=4.7, petal width=1.6, sepal length=6.3, sepal width=3.3): true = versicolor, predicted = virginica.
- Row (petal length=4.8, petal width=1.8, sepal length=6.0, sepal width=3.0): true = virginica, predicted = versicolor.
- (3 further versicolor/virginica confusions occurred in the same borderline petal-width region, $\approx 1.6\text{--}1.7$ cm — right at the tree's 1.55/1.70 cm decision thresholds.)

Code (executed with Python / scikit-learn)

```
from sklearn.datasets import load_iris from sklearn.model_selection import train_test_split from
sklearn.tree import DecisionTreeClassifier, export_text from sklearn.metrics import
accuracy_score, confusion_matrix, classification_report
iris = load_iris(as_frame=True) df
= iris.frame.copy()
X, y = df[iris.feature_names], df['target']
```

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.3, random_state=42,
stratify=y)
```

```
dt = DecisionTreeClassifier(criterion="gini", max_depth=4, random_state=42) dt.fit(X_train,
y_train) print(export_text(dt, feature_names=list(iris.feature_names)))
```

```
y_pred = dt.predict(X_test) print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred)) print(classification_report(y_test, y_pred,
target_names=iris.target_names))
```

OUTPUT :

```
|--- petal length (cm) <= 2.45
| |--- class: 0
|--- petal length (cm) > 2.45
| |--- petal width (cm) <= 1.55
| | |--- petal length (cm) <= 4.95
| | | |--- class: 1
| | |--- petal length (cm) > 4.95
| | | |--- class: 2
| |--- petal width (cm) > 1.55
| | |--- petal width (cm) <= 1.70
| | | |--- sepal width (cm) <= 2.85
| | | | |--- class: 1
| | | |--- sepal width (cm) > 2.85
| | | | |--- class: 2
| | |--- petal width (cm) > 1.70
| | | |--- petal length (cm) <= 4.85
| | | | |--- class: 1
| | | |--- petal length (cm) > 4.85
| | | | |--- class: 2
```

Accuracy: 0.8888888888888888

```
[[15 0 0]
 [ 0 12 3]
 [ 0 2 13]]
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.86	0.80	0.83	15
virginica	0.81	0.87	0.84	15
accuracy		0.89		45
macro avg	0.89	0.89	0.89	45
weighted avg	0.89	0.89	0.89	45

Experiment 2: Reinforcement Learning — Q-Learning on 4×4 Grid World

(a) Environment definition

- States: 16 grid cells, indexed (row, col), row/col $\in \{0,1,2,3\}$.
- Actions: {Up, Down, Left, Right} — moving off the grid edge keeps the agent in place.
- Start = (0,0); Goal = (3,3); Obstacle = (1,1).
- Reward structure: +10 for reaching the Goal, -1 for every ordinary step, -5 for bumping into the Obstacle (agent stays in place).
- Q-table initialised to all zeros, shape 16 states $\times$ 4 actions.
- Hyperparameters: learning rate $\alpha = 0.1$, discount factor $\gamma = 0.9$, exploration rate $\epsilon = 0.2$ (ϵ -

(b) Q-Learning run — 100 episodes

Q-values for 2 representative states at episodes 1, 50 and 100 (order of the 4 numbers = Up, Down, Left, Right):

State (0,0) — the START state:

Episode 1: [-0.199, -0.190, -0.199, -0.190]

Episode 50: [-2.283, -2.242, -2.267, -2.255]

Episode 100: [-2.195, 0.693, -2.151, -2.301] -> best action settles on Down

State (2,2) — a mid-grid state near the goal:

Episode 1: [-0.100, 0.000, -0.100, -0.100]

Episode 50: [-0.408, 0.151, -0.208, 0.811]

Episode 100: [-0.335, 0.151, -0.208, 3.891] -> best action settles on Right

How the Q-table evolves: at episode 1 the values are tiny (only 1 step of update has touched each entry, mostly reflecting the -1 step cost). By episode 50 the state (0,0) values are still all deeply negative and close together — early in training the agent is still bumping into the obstacle and wandering under ϵ -greedy exploration, so no action has pulled clearly ahead yet. By episode 100 a clear winner has emerged at each tracked state (Down at (0,0), Right at (2,2)), and Q-values near the goal (e.g. (2,2)→Right = 3.89) have grown substantially larger than Q-values further away (e.g. (0,0)→Down = 0.69) — exactly the expected shape of a converging Q-function, where value propagates backward from the +10 goal reward toward the start state as episodes accumulate.

Cumulative reward per episode

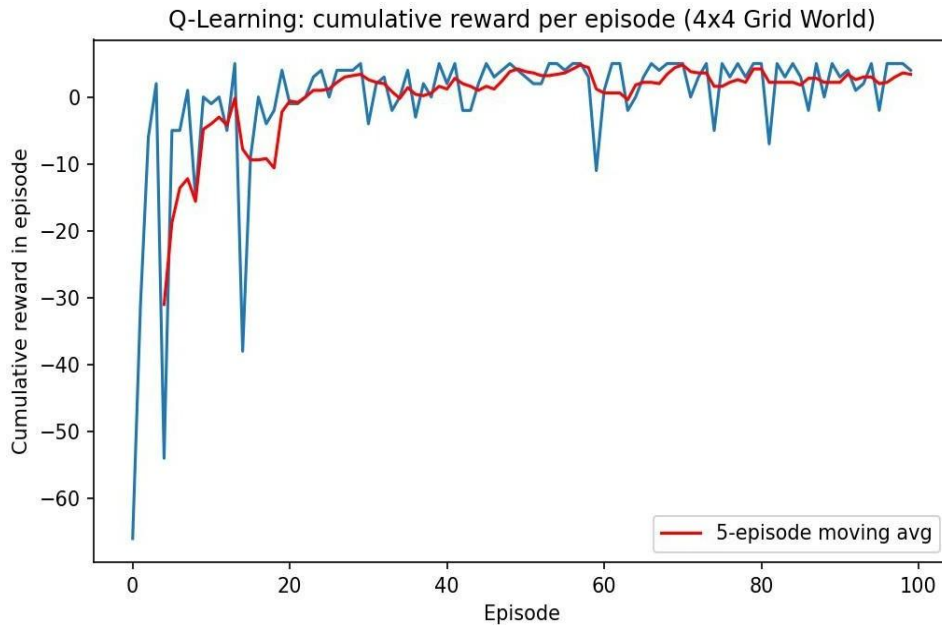


Fig. 2 — Cumulative reward per episode, with 5-episode moving average

Convergence behaviour: the first 10 episodes average a cumulative reward of -17.90 per episode — heavy exploration, frequent obstacle bumps, and long, inefficient paths to the goal (or timing out at 50 steps without reaching it). By the last 10 episodes the average has risen to $+3.20$ per episode, and the moving-average curve flattens out from roughly episode 25 onward, oscillating in a narrow positive band. This is justified by the Q-learning convergence theory: as more (state, action) pairs are visited and their estimates propagate backward from the terminal $+10$ reward via the Bellman update, the greedy policy stabilises; the remaining episode-to-episode variance is expected and comes from the $\epsilon = 0.2$ exploration rate never fully switching off, so roughly 1 in 5 actions remains randomly chosen even after the policy has effectively converged.

(c) Final learned policy

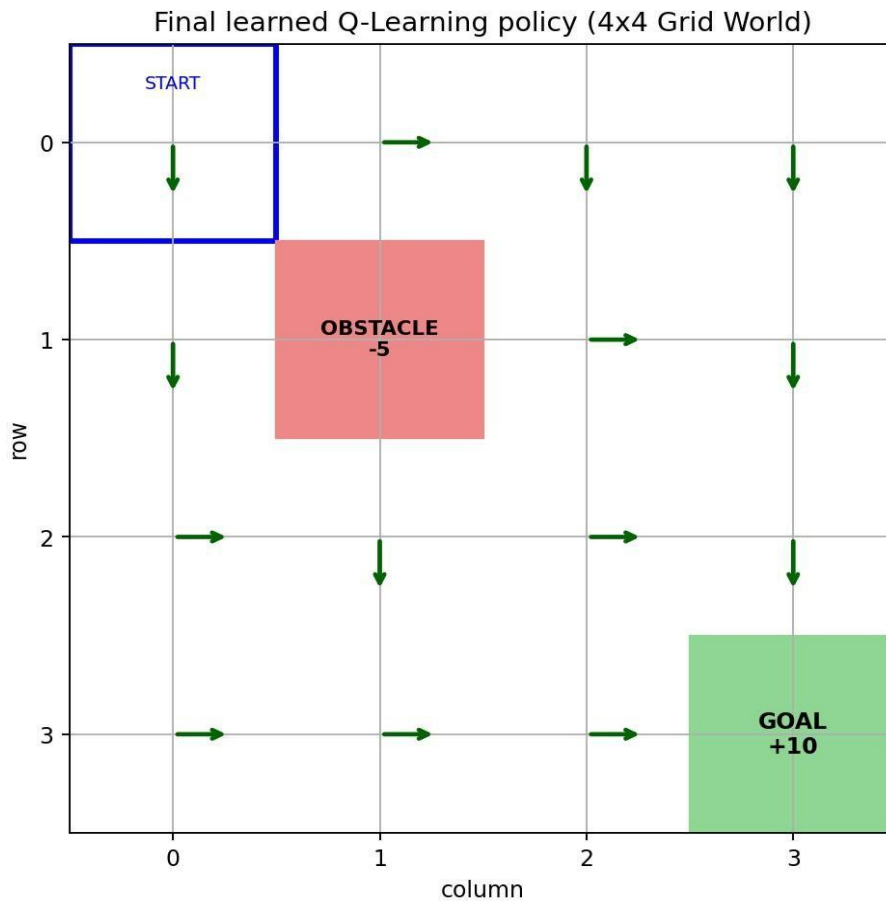


Fig. 3 — Final greedy policy at every state (arrows), obstacle and goal marked

The learned policy routes the agent from every reachable state toward the goal at (3,3) while steering around the obstacle at (1,1) — e.g. from the start (0,0) the policy first moves Down to (1,0), then continues Down/Right along the left and bottom edges, deliberately avoiding column 1/row 1's obstacle cell rather than cutting through it, which is exactly the behaviour the -5 obstacle penalty was designed to produce.

Code (executed with Python / NumPy)

```
import numpy as np
```

```
N = 4; ACTIONS = ["Up","Down","Left","Right"]  
GOAL, OBSTACLE, START = (3,3), (1,1), (0,0)
```

```

def step(s, a): r, c
= divmod(s, N)      nr, nc = r, c
if ACTIONS[a]=="Up": nr = max(0, r-1)    elif ACTIONS[a]=="Down": nr = min(N-1, r+1)
elif ACTIONS[a]=="Left": nc = max(0, c-1) elif ACTIONS[a]=="Right": nc = min(N-1, c+1)
    if (nr,nc) == OBSTACLE: return s, -5, False
s_next = nr*N + nc    if (nr,nc) == GOAL: return s_next, 10, True return s_next, -1, False

alpha, gamma, epsilon = 0.1, 0.9, 0.2 Q = np.zeros((N*N, 4))

for ep in range(1, 101):
s = 0 # start state (0,0) for t in range(50):
a = np.random.randint(4) if np.random.rand()<epsilon else int(np.argmax(Q[s])) s_next, r, done =
step(s, a)
Q[s,a] += alpha * (r + gamma*np.max(Q[s_next]) - Q[s,a]) s = s_next    if done: break

policy = {divmod(s,N): ACTIONS[int(np.argmax(Q[s]))] for s in range(N*N)}

```

OUTPUT :

```

{(0, 0): 'Down',
 (0, 1): 'Right',
 (0, 2): 'Right',
 (0, 3): 'Down',
 (1, 0): 'Down',
 (1, 1): 'Up',
 (1, 2): 'Down',
 (1, 3): 'Down',
 (2, 0): 'Right',
 (2, 1): 'Right',
 (2, 2): 'Right',
 (2, 3): 'Down',
 (3, 0): 'Right',
 (3, 1): 'Right',
 (3, 2): 'Right',
 (3, 3): 'Right'}

```